



CS-2001
Data Structures
Fall 2023

Binary Heap

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Motivation

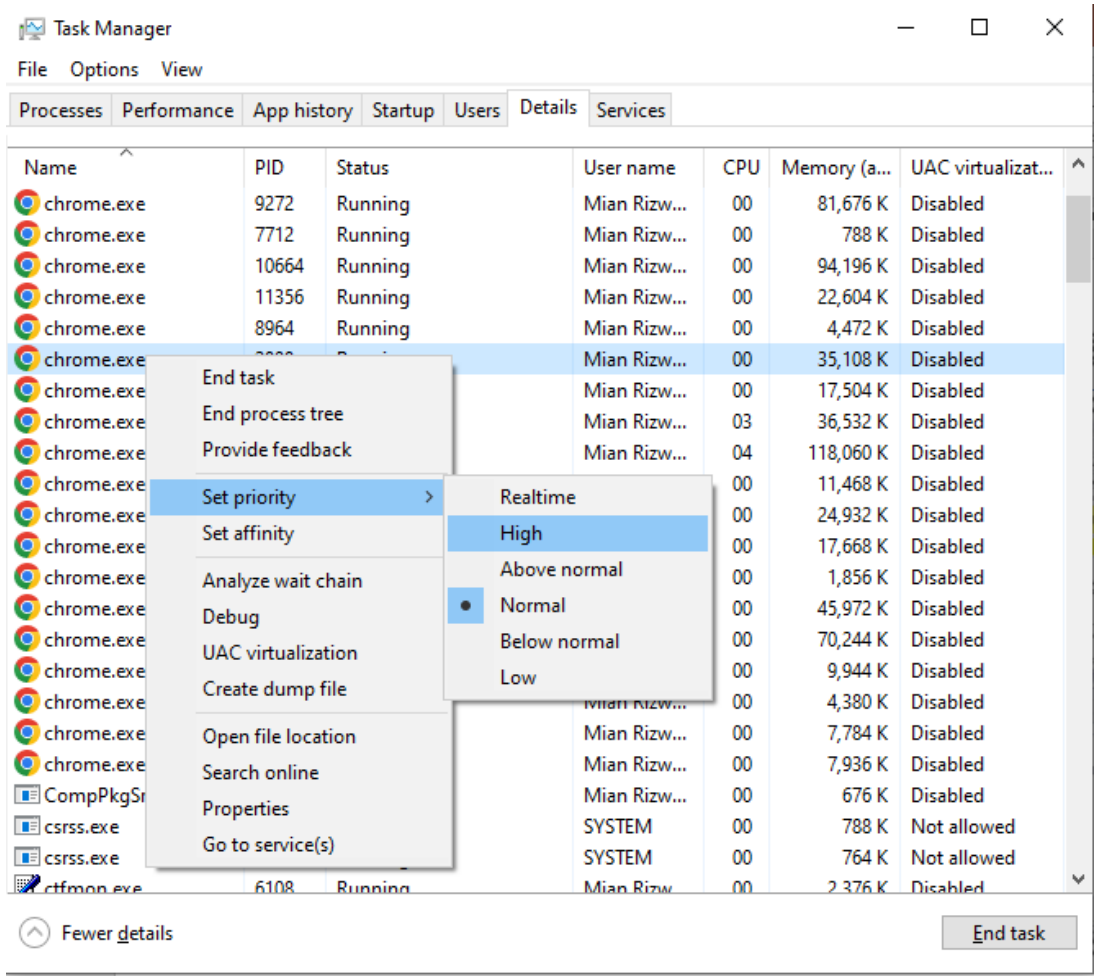
- With **queues** the order may be summarized by **first in, first out**

- Some tasks may be more important or timely than others
  - Higher priority

- Priority queues
  - Enqueue objects based on priority
  - Dequeue that object which has highest priority
  - If two objects have same priority, they are served as [usually] FIFO order

Applications Of Priority Queue

- Store packets on network routers in order of urgency
- Ordering CPU jobs
- Hospital emergency-room admission processing
- Dijkstra algorithm, Prim's algorithm for MST, and A* algorithms in AI use priority queue to complete their operations



The priority of processes in Windows may be set in the Windows Task Manager

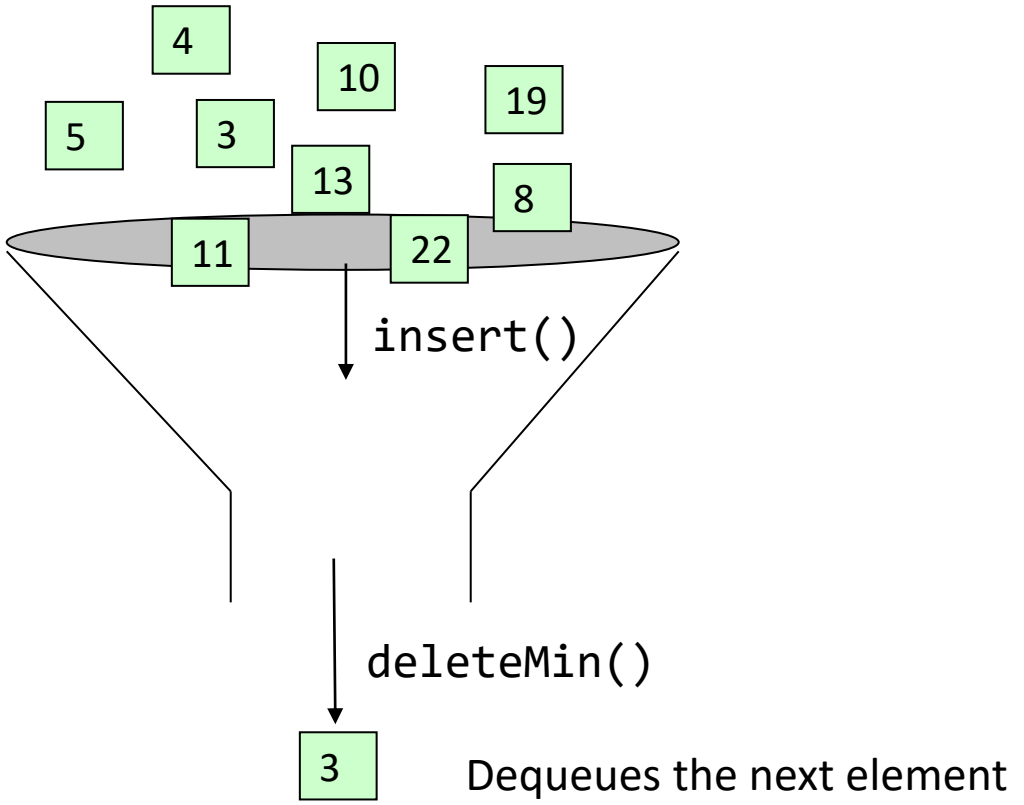
Priority Queue – ADT

- **insert** (i.e., enqueue)
  - Specification of a priority level (0-high, 1,2.. Low)

- **deleteMin** (i.e., dequeue)
  - Returns the current “highest priority” element in the queue
    - Element with the minimum priority level
  - Deletes that element from the queue

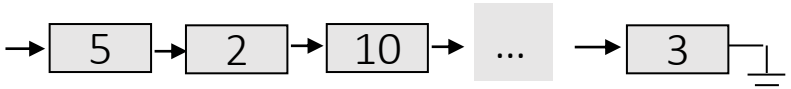
- Performance goal is to make the run time of each operation as close to O(1) as possible

Priority Queue – ADT

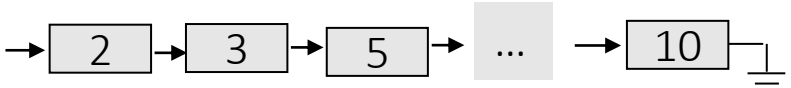


Simple Implementations

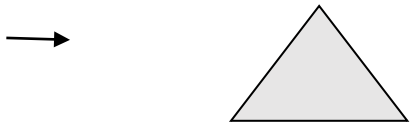
- Unordered linked list
  - `Insert` – `O(1)` step
  - `deleteMin` – `O(n)` steps



- Ordered linked list
  - `insert` – `O(n)` steps
  - `deleteMin` – `O(1)` step



- Balanced binary tree,
  - `insert` – $O(\log_2 n)$ steps
  - `deleteMin` in how many steps?
    - Find min – $O(\log_2 n)$ steps
    - Delete – $O(\log_2 n)$ steps



Can we build a data structure better suited to store and retrieve priorities?

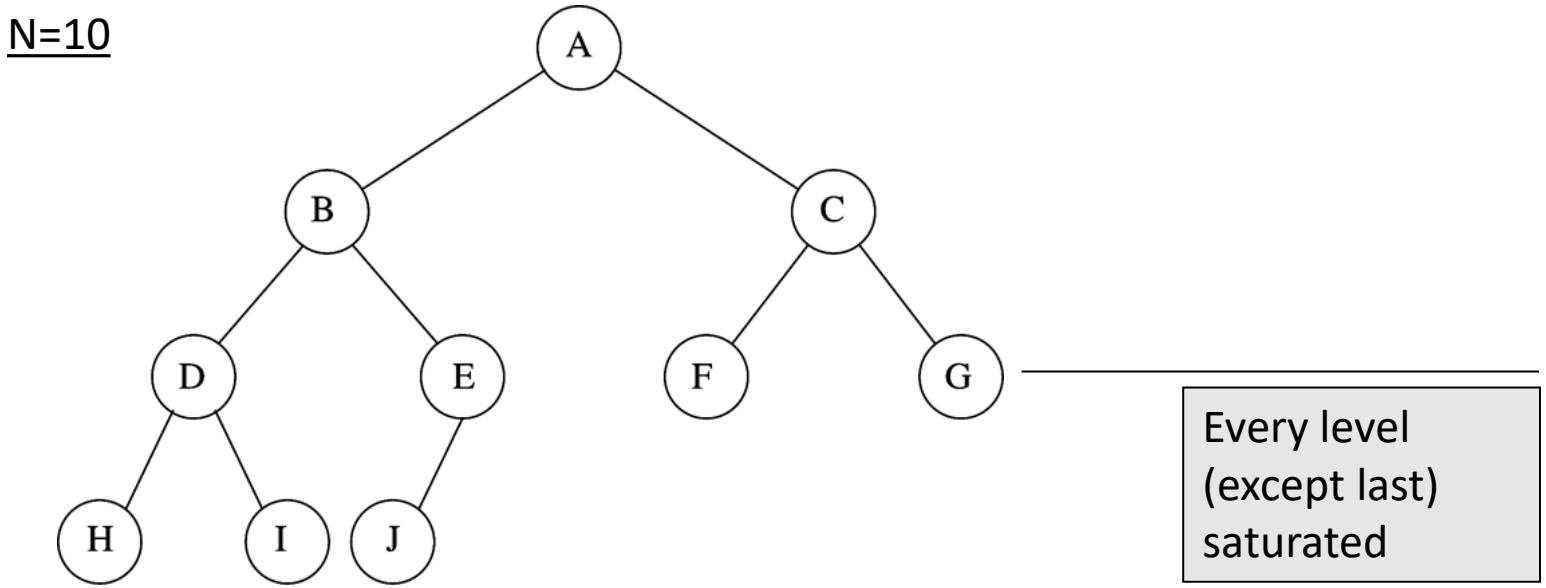
Binary Heap

Binary Heap

- A binary heap is a binary tree with two properties
  - Structure property
  - Heap-order property

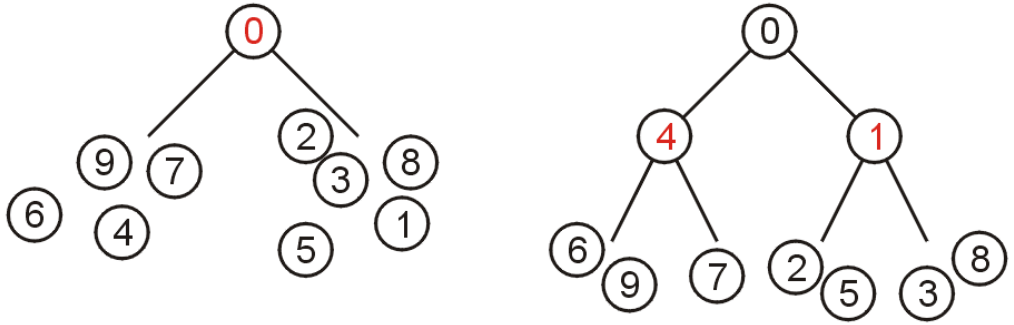
Binary Heap – Structure Property

- A **binary heap** is **(almost) complete** binary tree
  - Each level (except possibly the bottom most level) is completely filled
  - The bottom most level may be partially filled (from left to right



Binary Heap – Heap-Order Property

- **Min-Heap** property
  - Key associated with the root is less than or equal to the keys associated with either of the sub-trees (if any)
  - Both of the sub-trees (if any) are also binary min-heaps



- **Properties** of min-heap
  - A single node is a min-heap
  - **Minimum** key always at **root**
  - For every node X, key(parent(X)) ≤ key(X)
  - **No relationship** between nodes with **similar key**

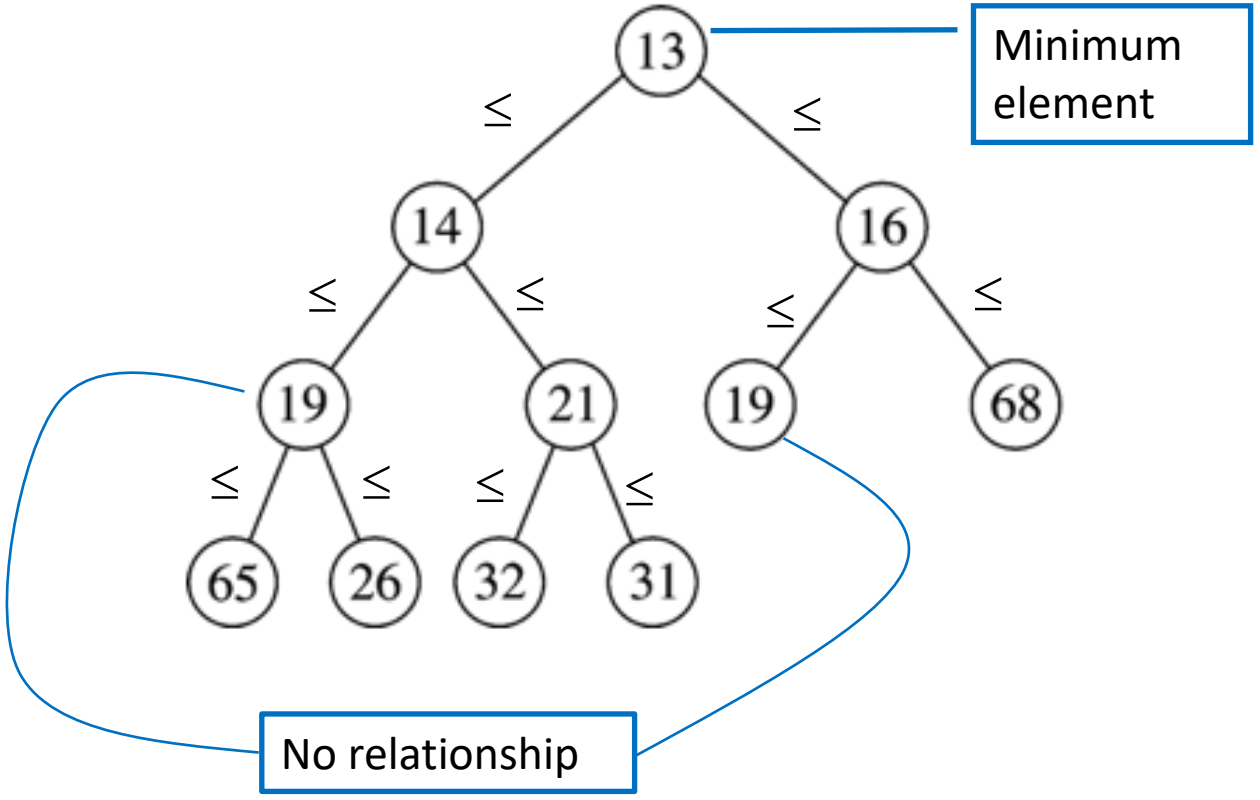
Binary Heap – Heap-Order Property

- Max-Heap property
  - Maximum key at the root
  - For every node X, key(parent(X)) ≥ key(X)

- Insert and deleteMin must maintain heap-order property

Heap-Order Property – Example

- Min-Heap



Maintaining the heap property

BUILD_Max_HEAP*(A)*
1 *heap_size[A] ← length[A]*
2 **for** *i ← heap_size[A] / 2 downto 1*
3 **do** MAX_HEAPIFY*(A, i)*

MAX_HEAPIFY*(A, i)*
1 *L ← 2i*
2 *R ← 2i + 1*
3 **if** *L ≤ heap_size[A] and A[L] > A[i]*
4 **then** *max ←L*
5 **else** *max ←i*

6 **if** *R ≤ heap_size[A] and A[R] > A[max]*
7 **then** *max ←R*
8 **if** *max != i*
9 **then** exchange *A[i] ↔ A[max]*
10 MAX_HEAPIFY*(A, max)*

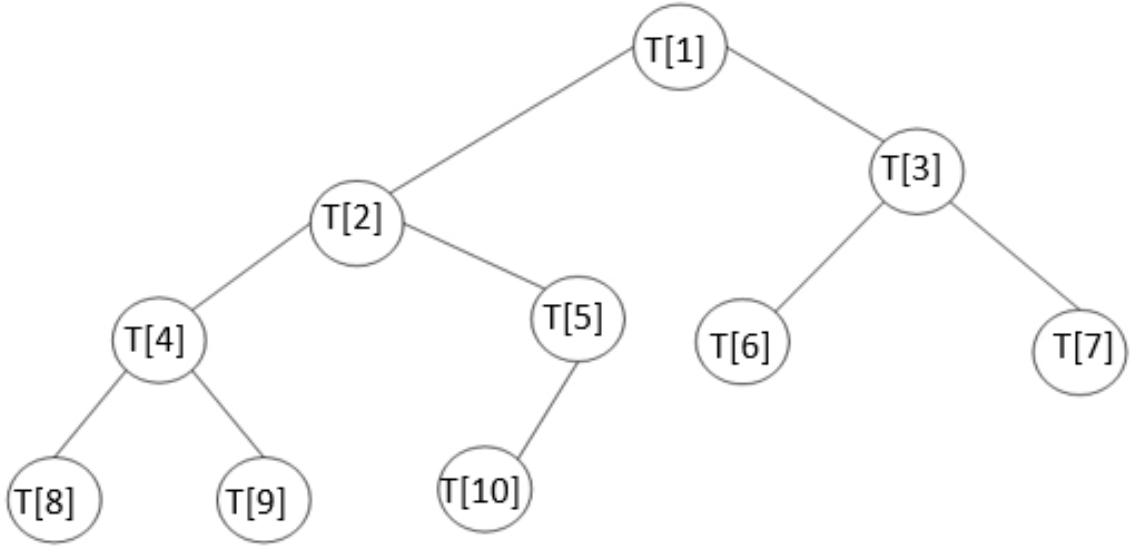
Maintaining the heap property

BUILD_Min_HEAP*(A)*
1 *heap_size[A] ← length[A]*
2 **for** *i ← heap_size[A] / 2 downto 1*
3 **do** MIN_HEAPIFY*(A, i)*

MIN_HEAPIFY*(A, i)*
1 *L ← 2i*
2 *R ← 2i + 1*
3 **if** *L ≤ heap_size[A] and A[L] < A[i]*
4 **then** *min ←L*
5 **else** *min ←i*

6 **if** *R ≤ heap_size[A] and A[R] < A[min]*
7 **then** *min ←R*
8 **if** *min != i*
9 **then** exchange *A[i] ↔ A[min]*
10 MIN_HEAPIFY*(A, min)*

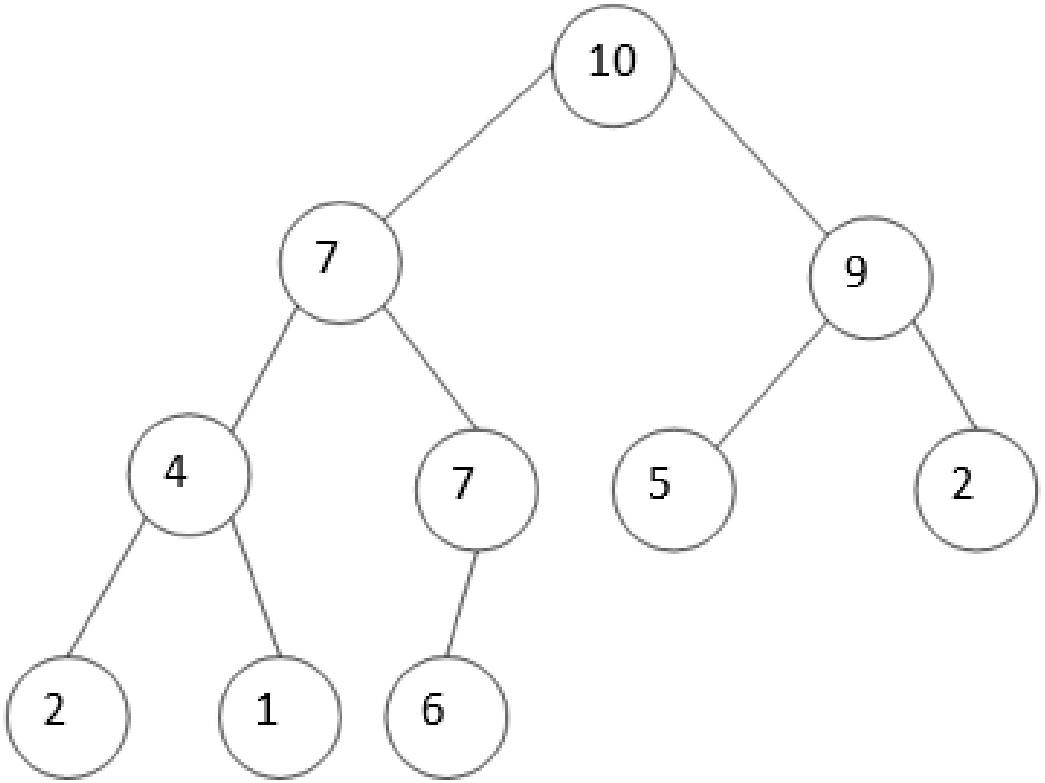
An al-most complete binary tree



- A complete binary tree containing 10 nodes.
- Five internal nodes occupy level 3 (the root), level 2, and the left side of level 1.
- Five leaves fill the right side of level 1 and then continue at the left of level 0.

A heap

- Now we have marked each node with its value

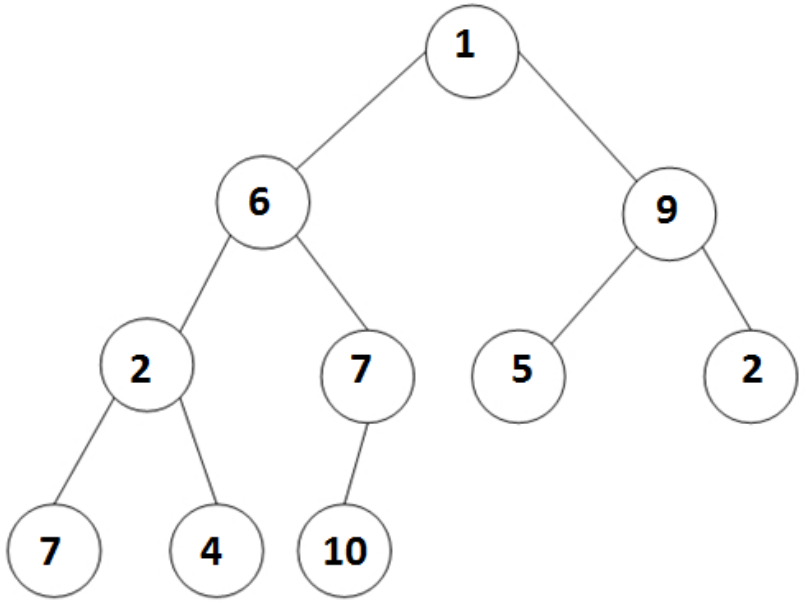


Making Heaps

- There exists a clever algorithm for making a heap. Suppose, for example, that our starting point is the following array represented by the tree in Figure.

-	1	6	9	2	7	5	2	7	4	10
0	1	2	3	4	5	6	7	8	9	10

- The starting situation



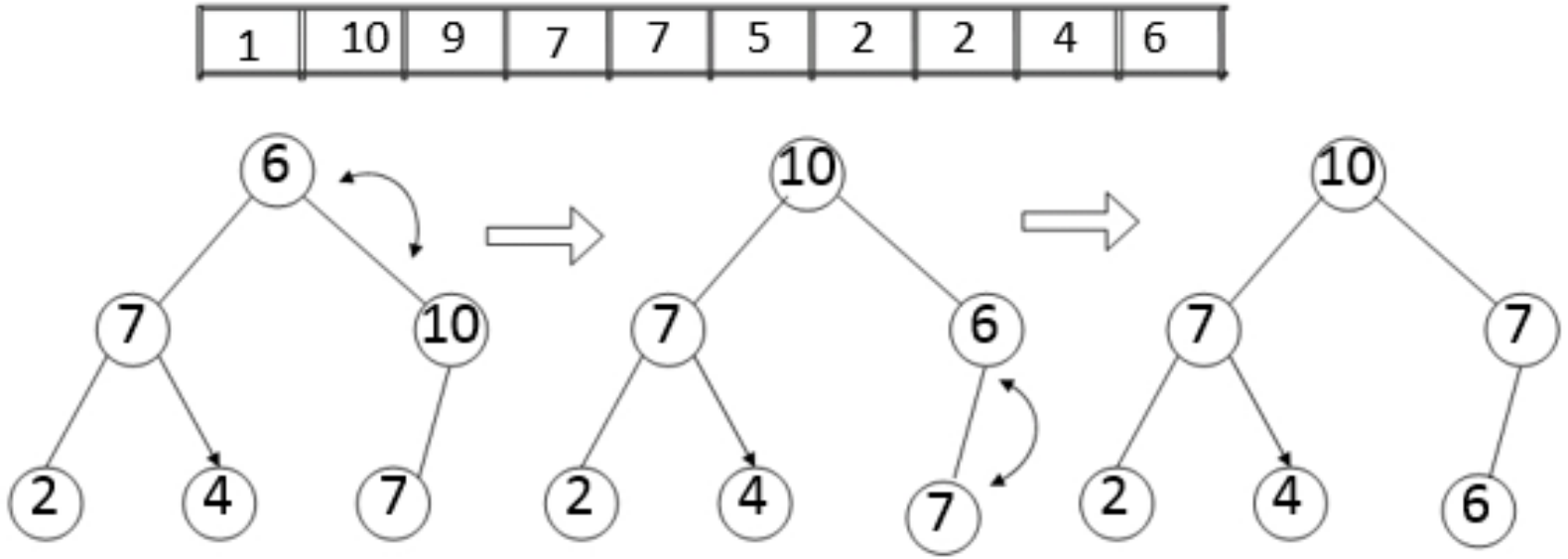
Making Heap

- We first make each of the subtrees whose roots are at level 1 into a heap, this is done by sifting down these nodes, as illustrated in Figure



Making Heap

- This figure shows the process for the left subtree. The other subtree at level 2 is already a heap. This results in an essentially complete binary tree corresponding to the array.

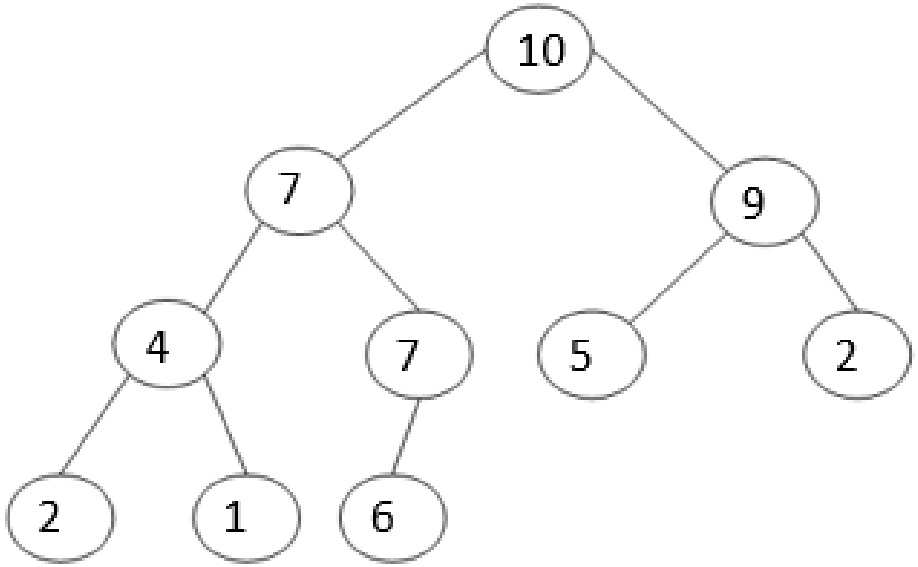


One level 2 subtree is made into a heap (the other already is a heap)

Making Heap

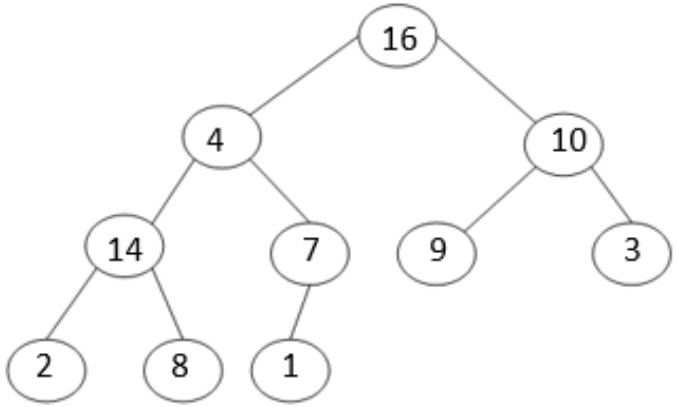
- It only remains to sift down its root to obtain the desired heap
- This process thus goes as follows:

10	1	9	7	7	5	2	2	4	6
10	7	9	1	7	5	2	2	4	6
10	7	9	4	7	5	2	2	1	6



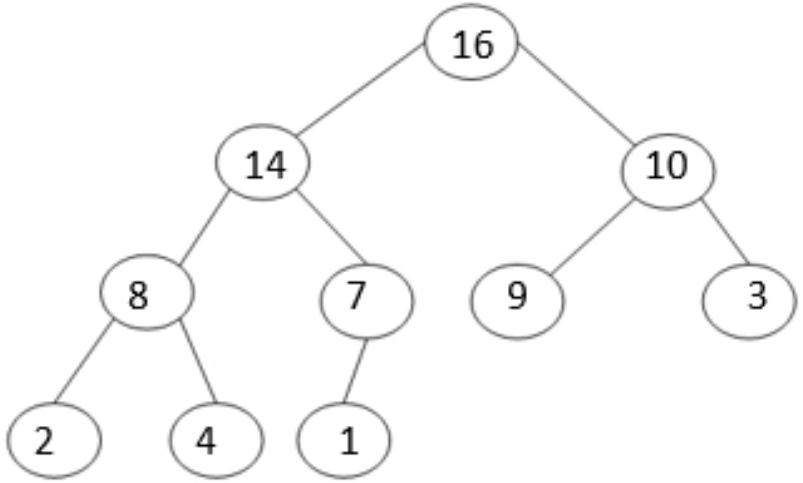
Exercise

- Construct the heap using the array A=(16, 4, 10, 14, 7, 9, 3, 2, 8, 1)



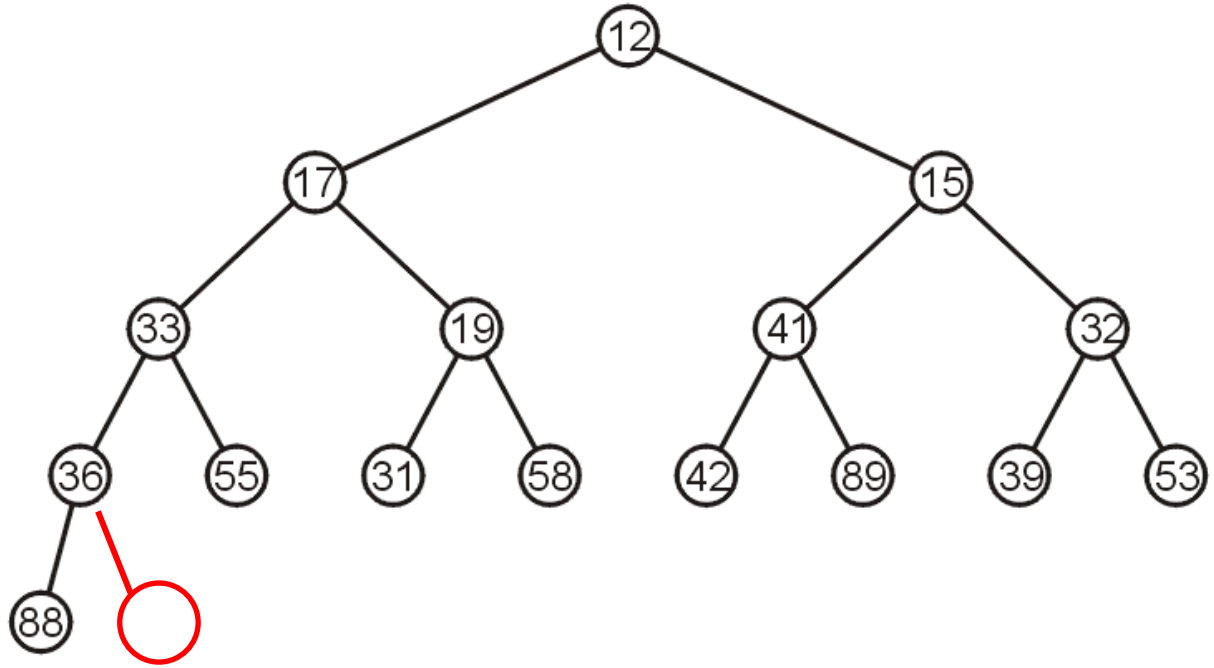
The initial configuration

Maintaining heap property



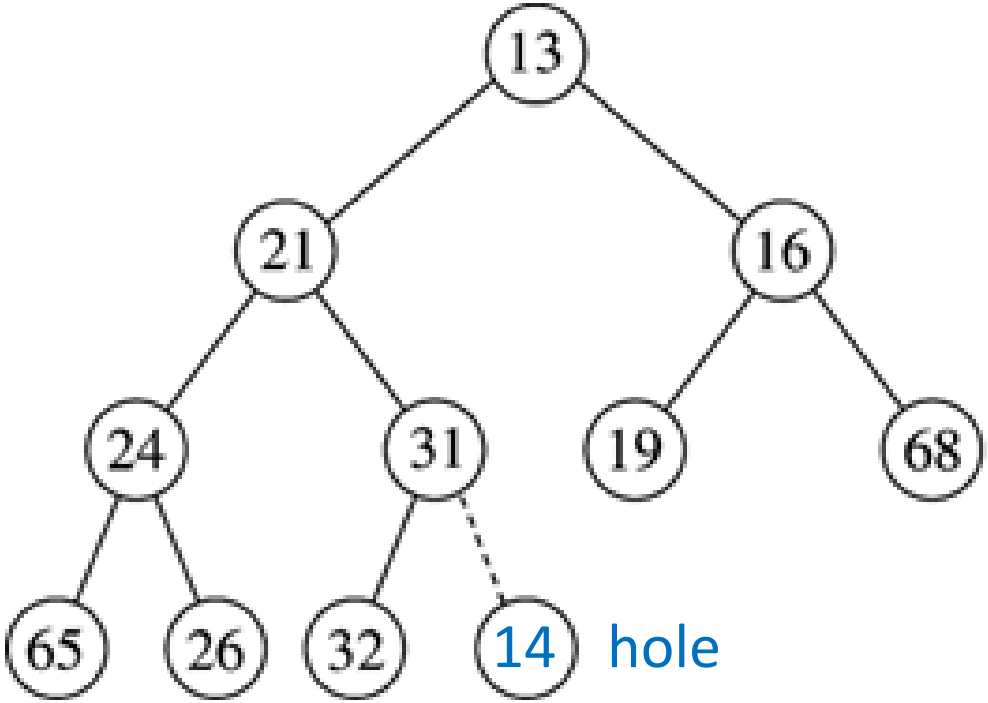
Heap Operations – `insert`

- Insert new element into the heap at the next available slot ("hole")
  - Maintaining (almost) complete binary tree
- **Percolate** the element **up** the heap while heap-order property not satisfied



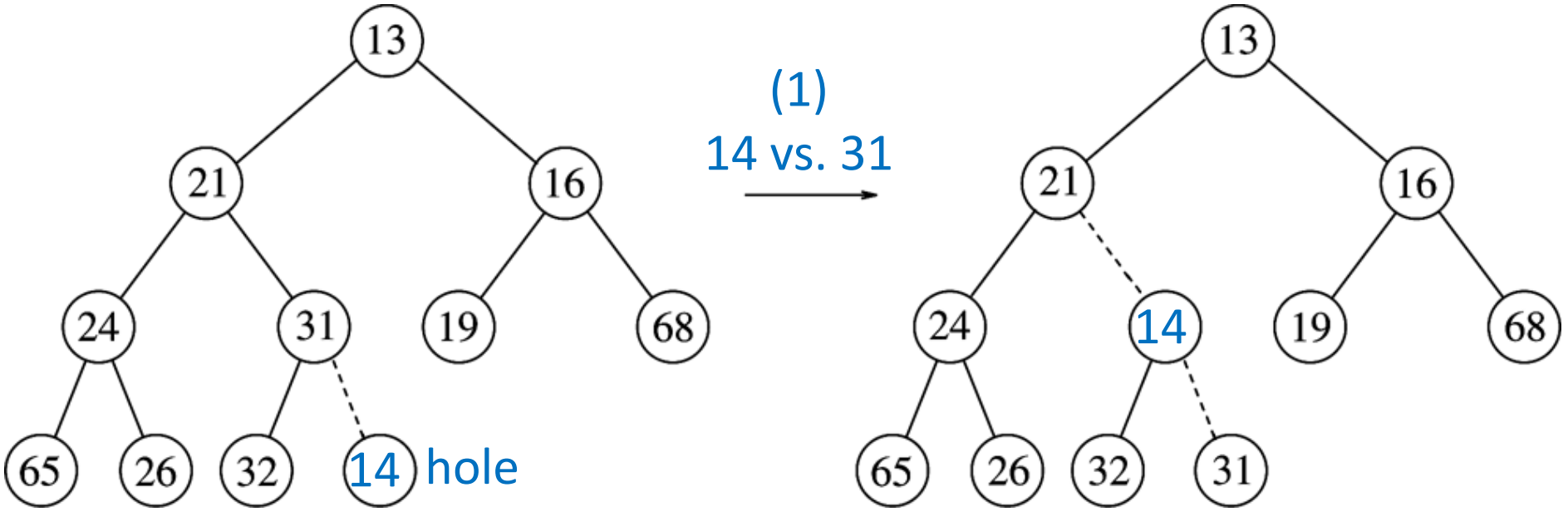
Heap Insert – Example

- Insert 14



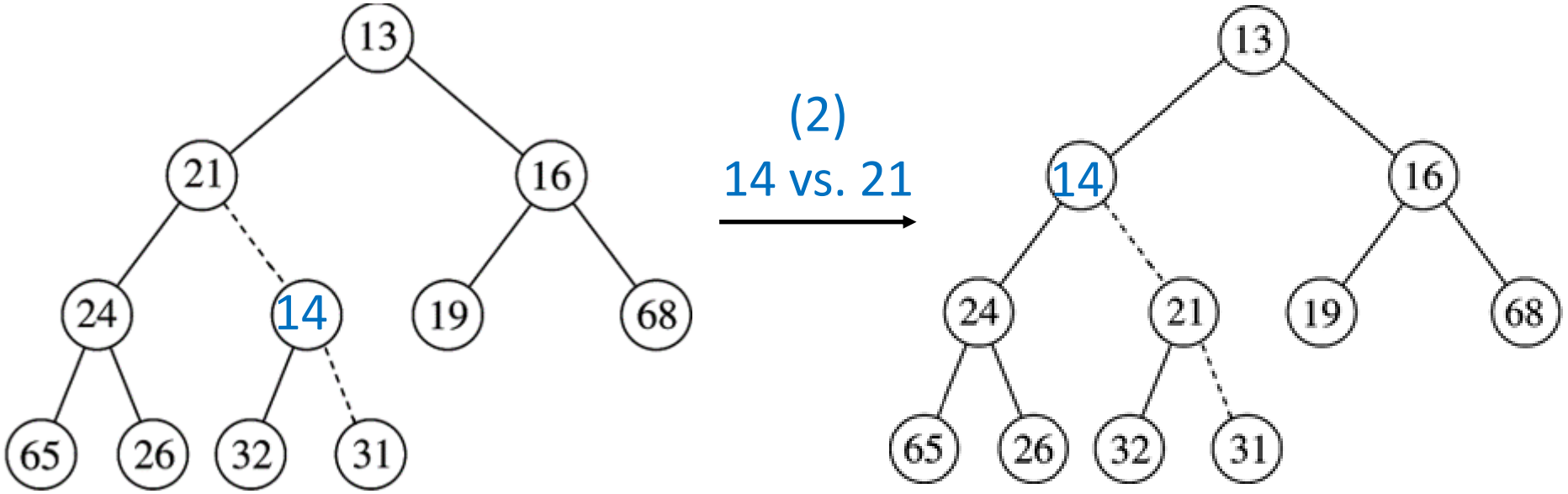
Heap Insert – Example

- Insert 14



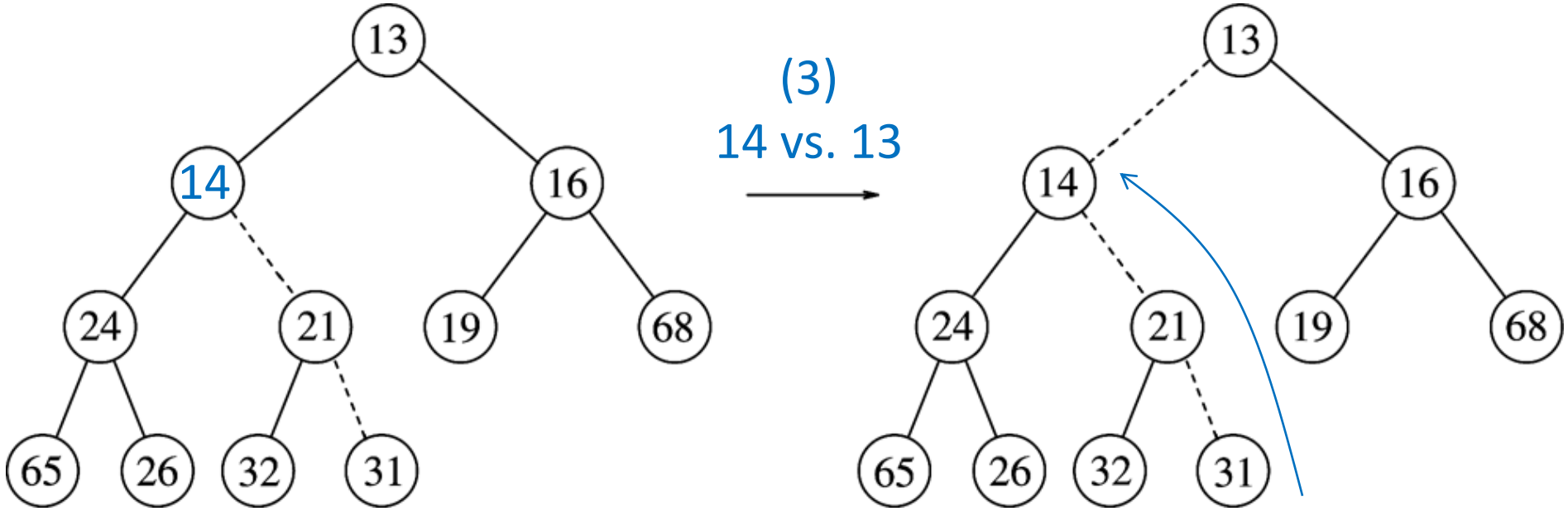
Heap Insert – Example

- Insert 14



Heap Insert – Example

- Insert 14

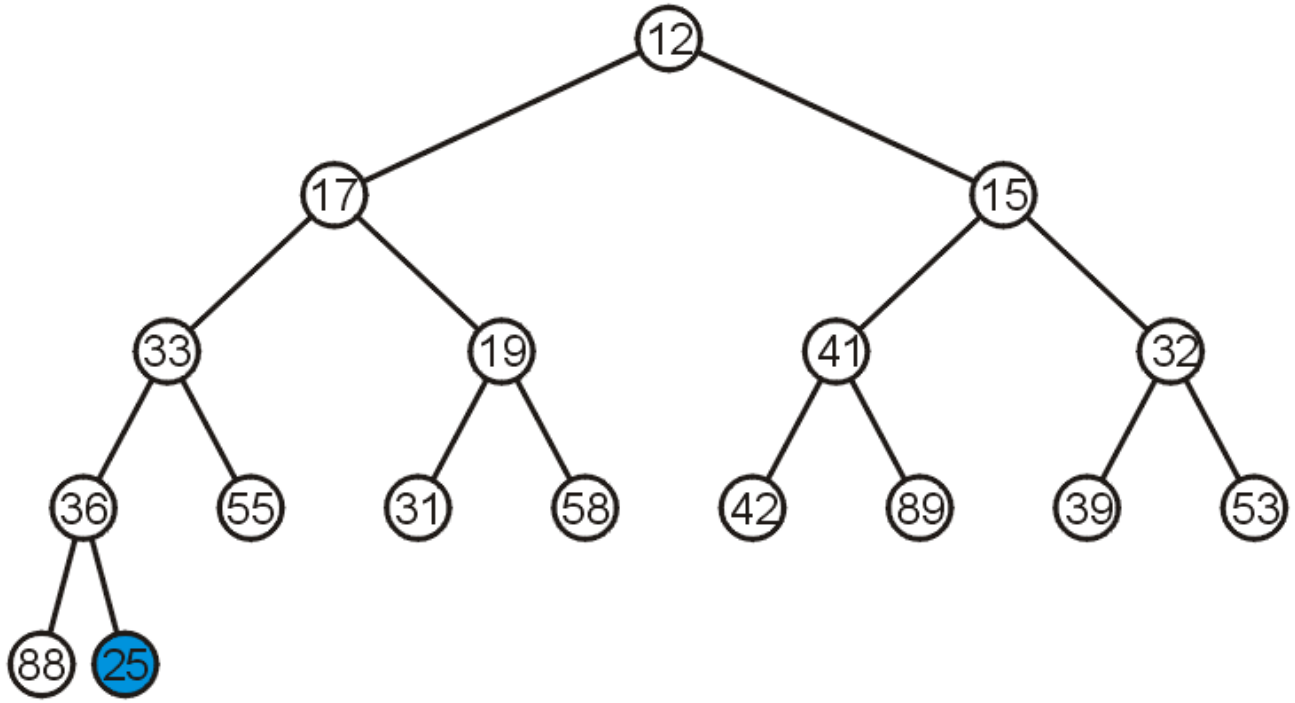


Heap order property

Structure property

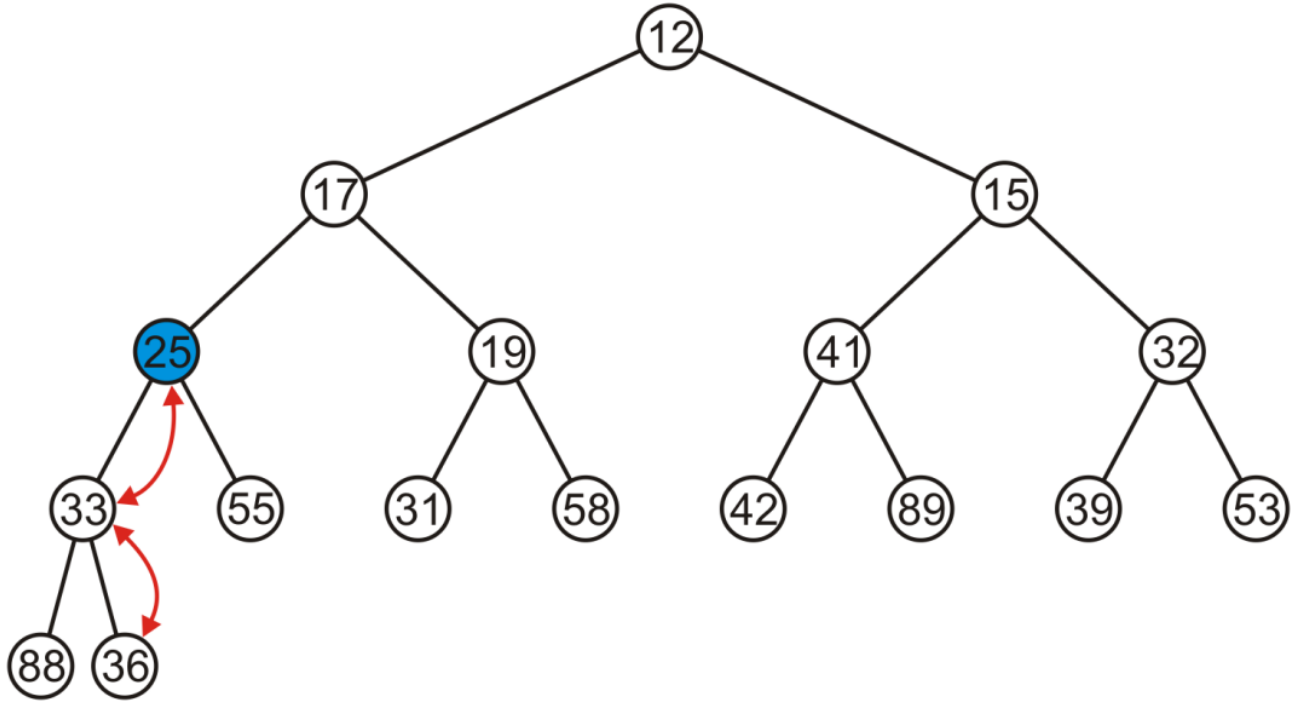
Heap Insert – Example

- Insert 25



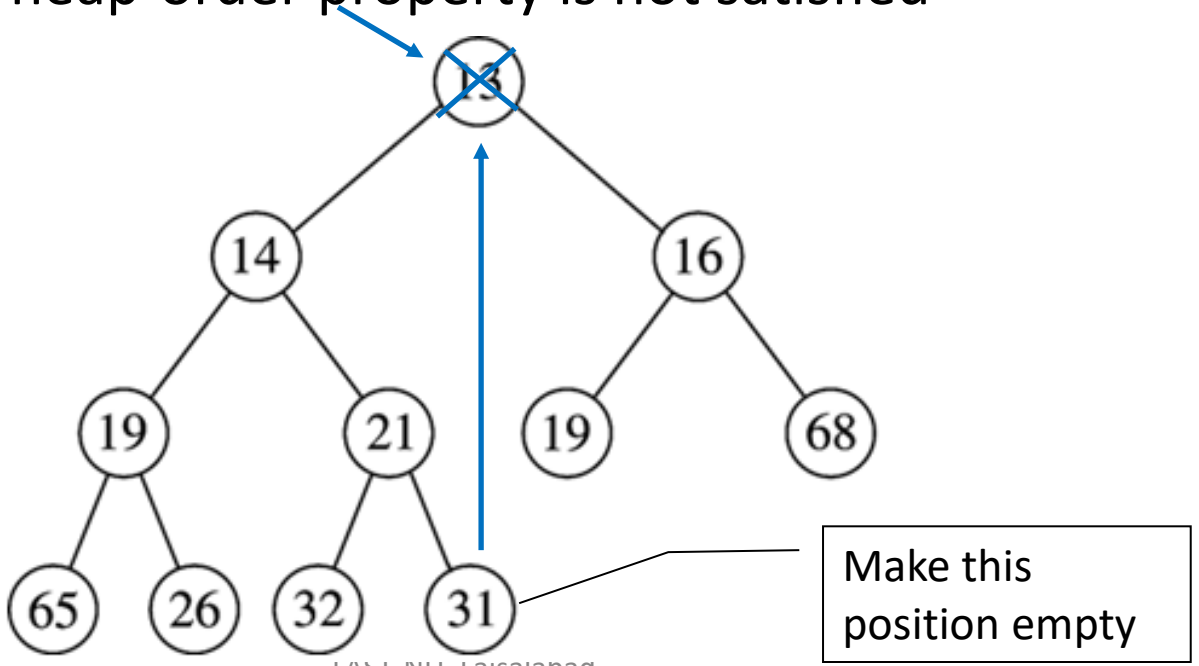
Heap Insert – Example

- Percolate 25 up into its appropriate location
  - The resulting heap is still a complete tree

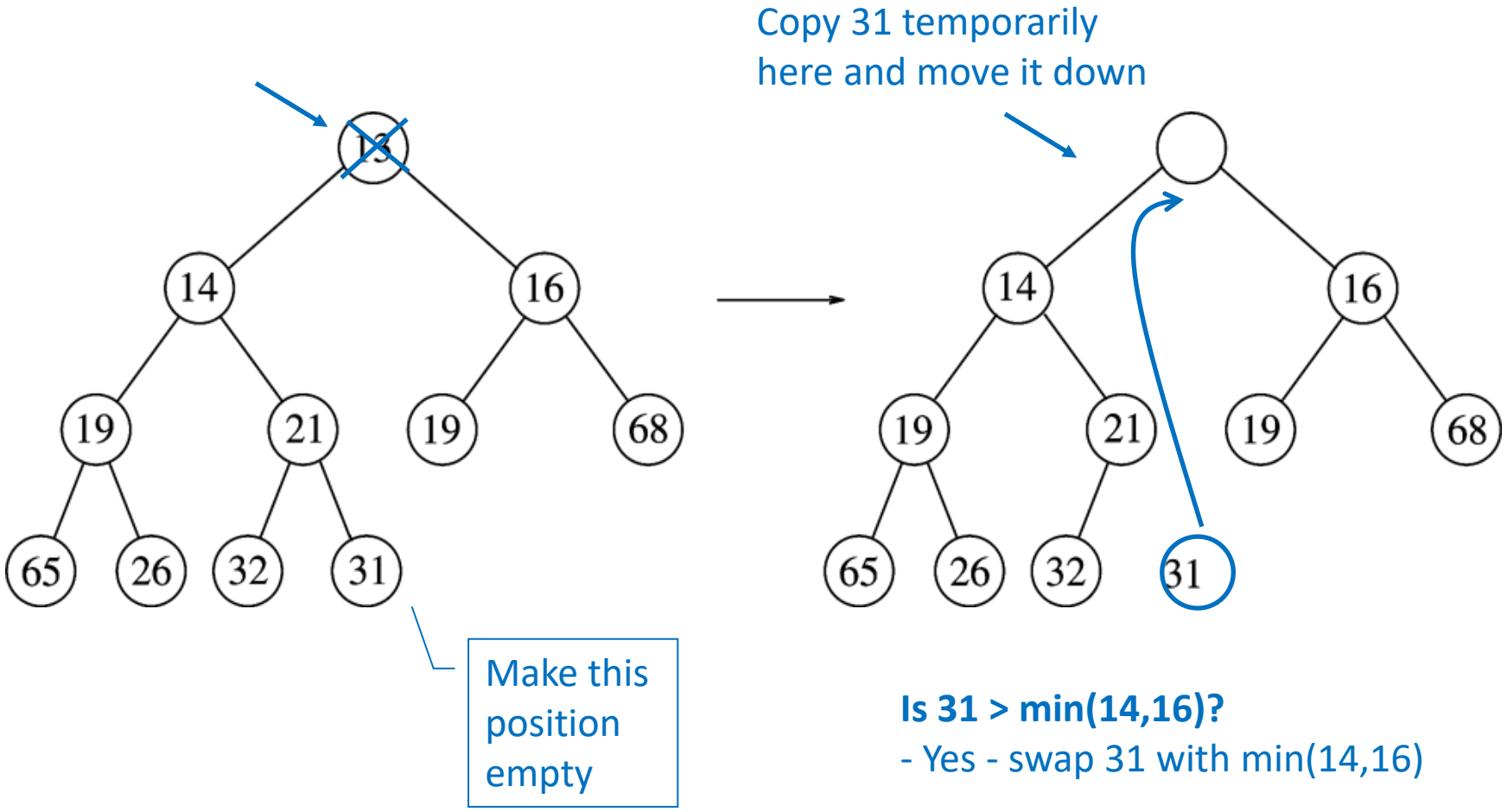


Heap Operation – `deleteMin`

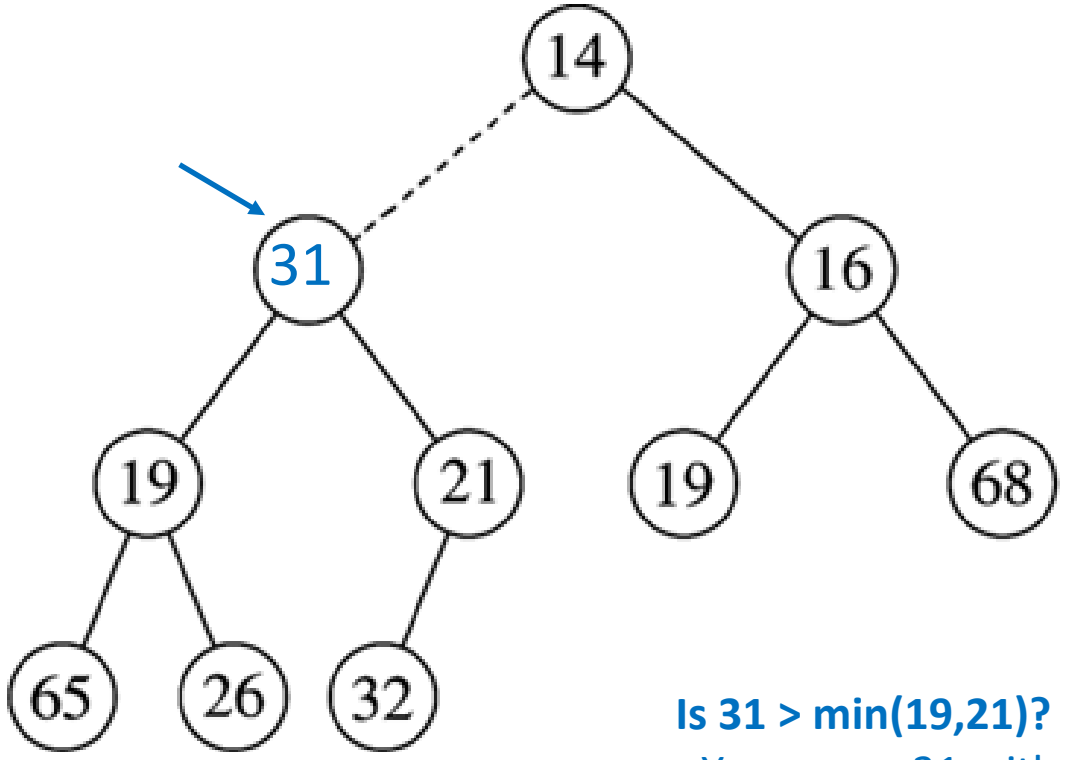
- Minimum element is always at the root
  - Return the element at the root and delete it
- Heap decreases by one in size
- Move last element of the tree into hole at root
- **Percolate down** while heap-order property is not satisfied



`deleteMin` – Example



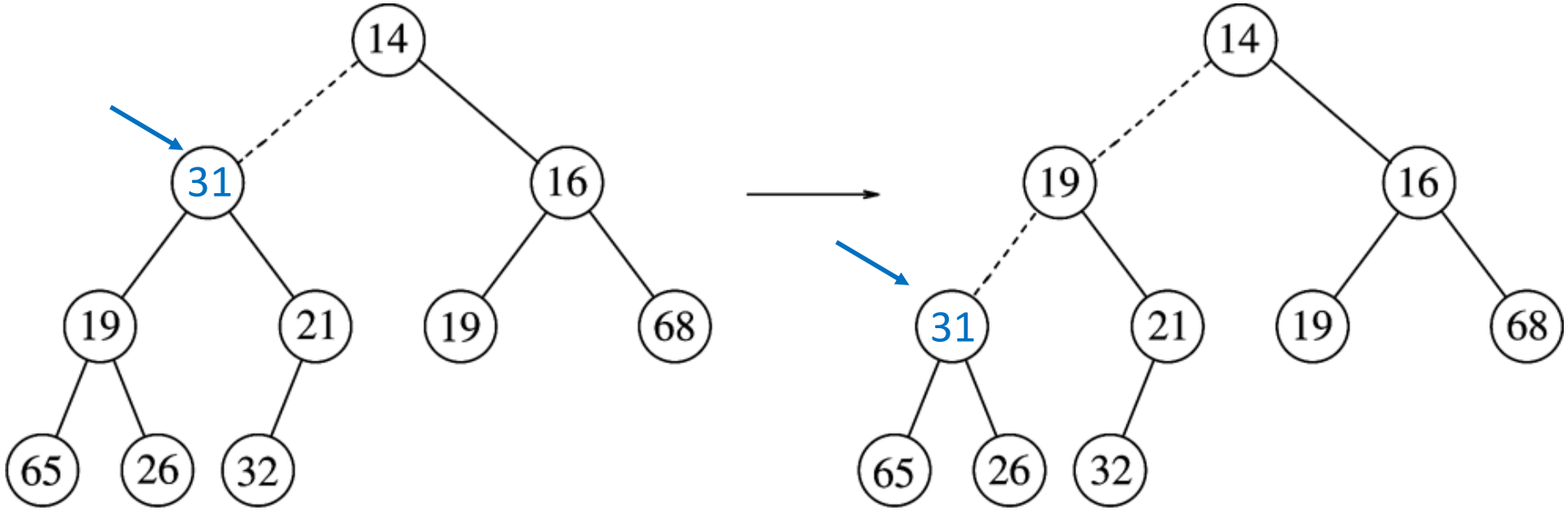
`deleteMin` – Example



Is 31 > min(19,21)?

- Yes - swap 31 with min(19,21)

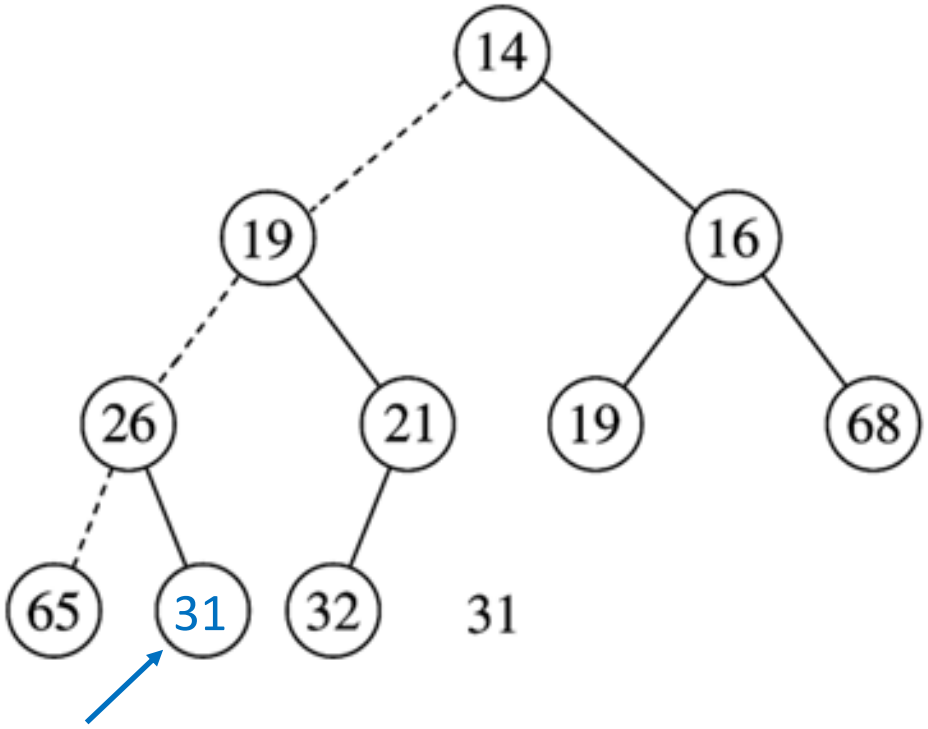
`deleteMin` – Example



Is 31 > min(19,21)?
- Yes - swap 31 with min(19,21)

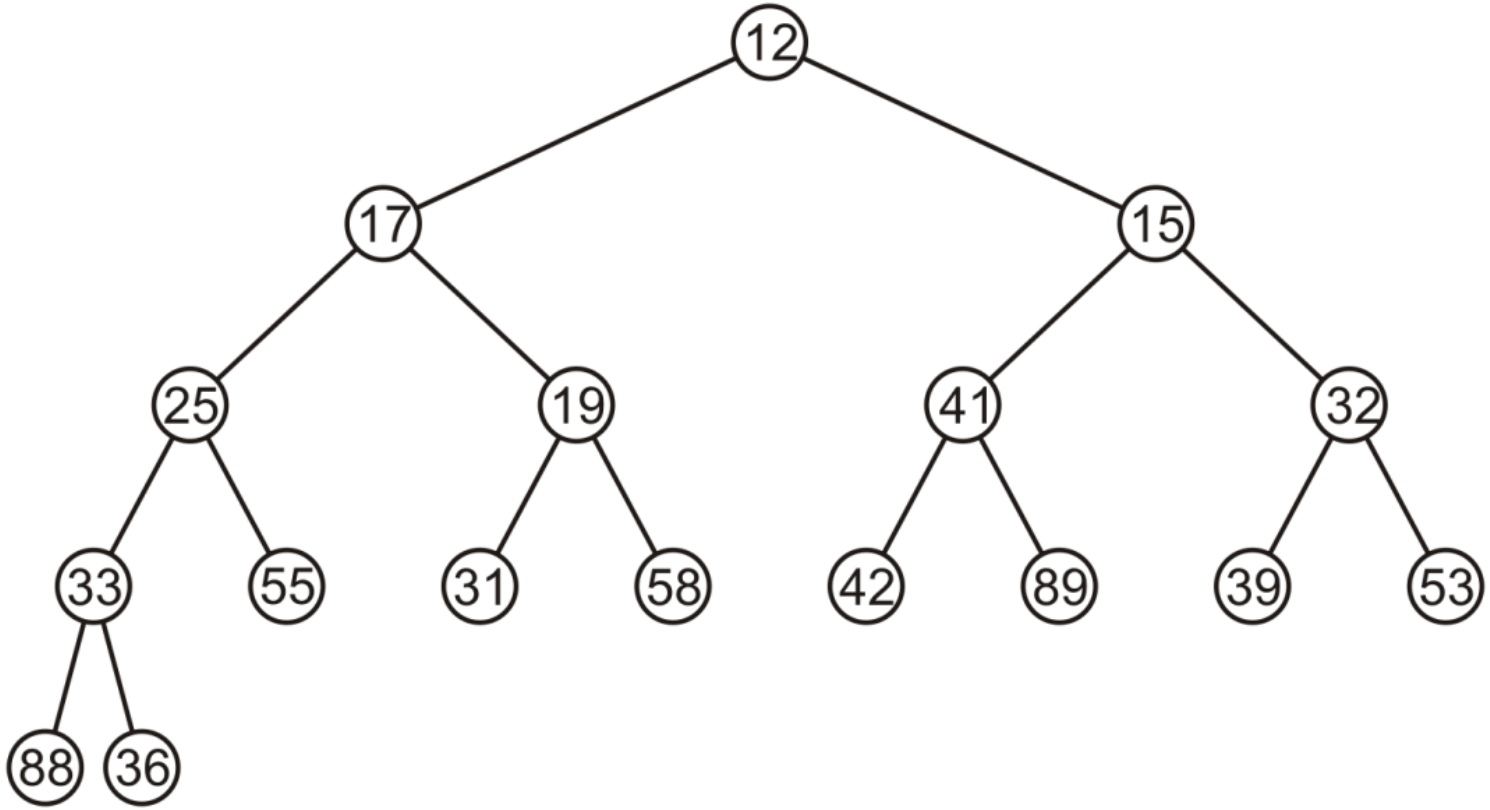
Is 31 > min(65,26)?
- Yes - swap 31 with min(65,26)

`deleteMin` – Example



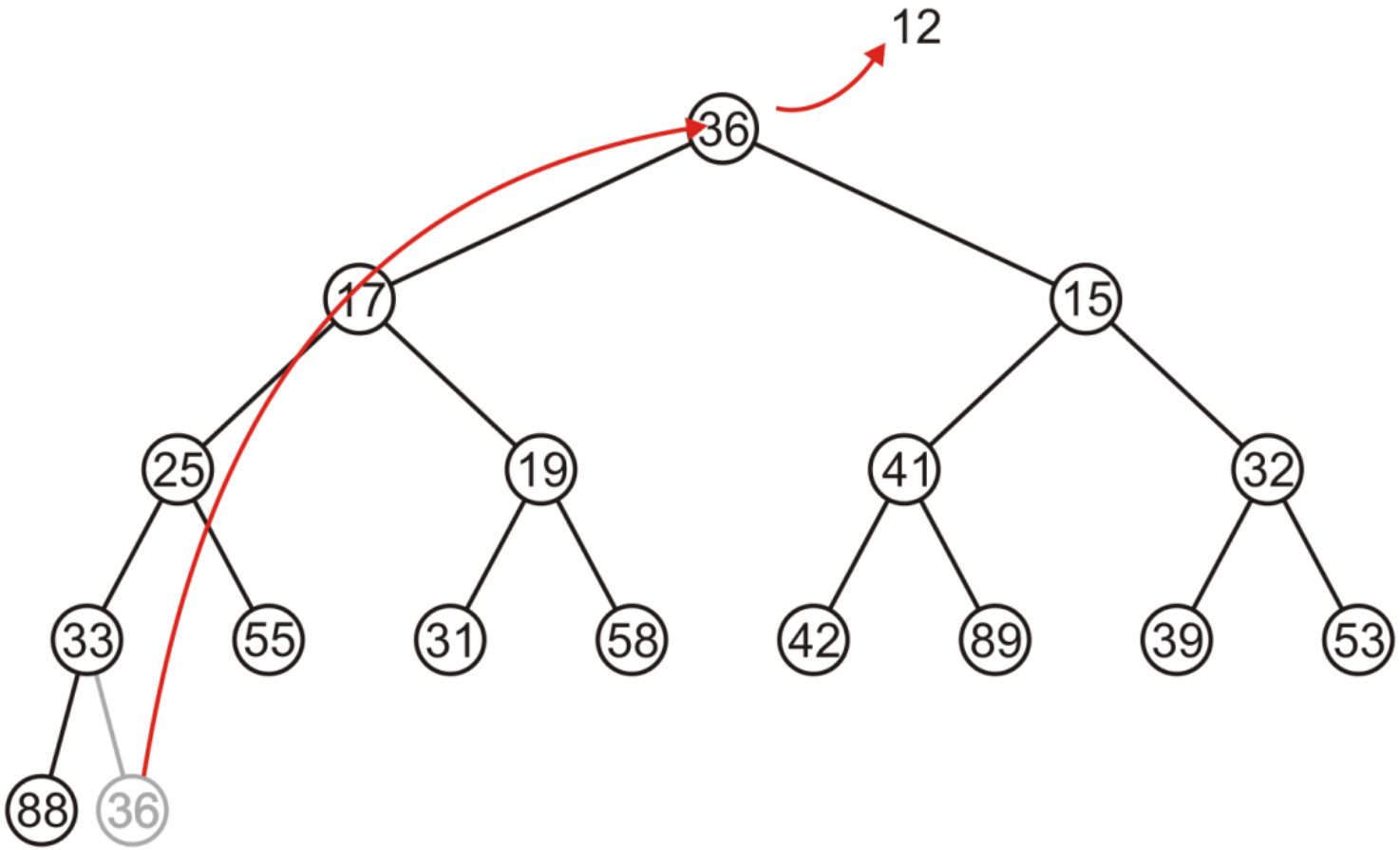
`deleteMin` – Example

- `deleteMin` will dequeue element 12 from the top



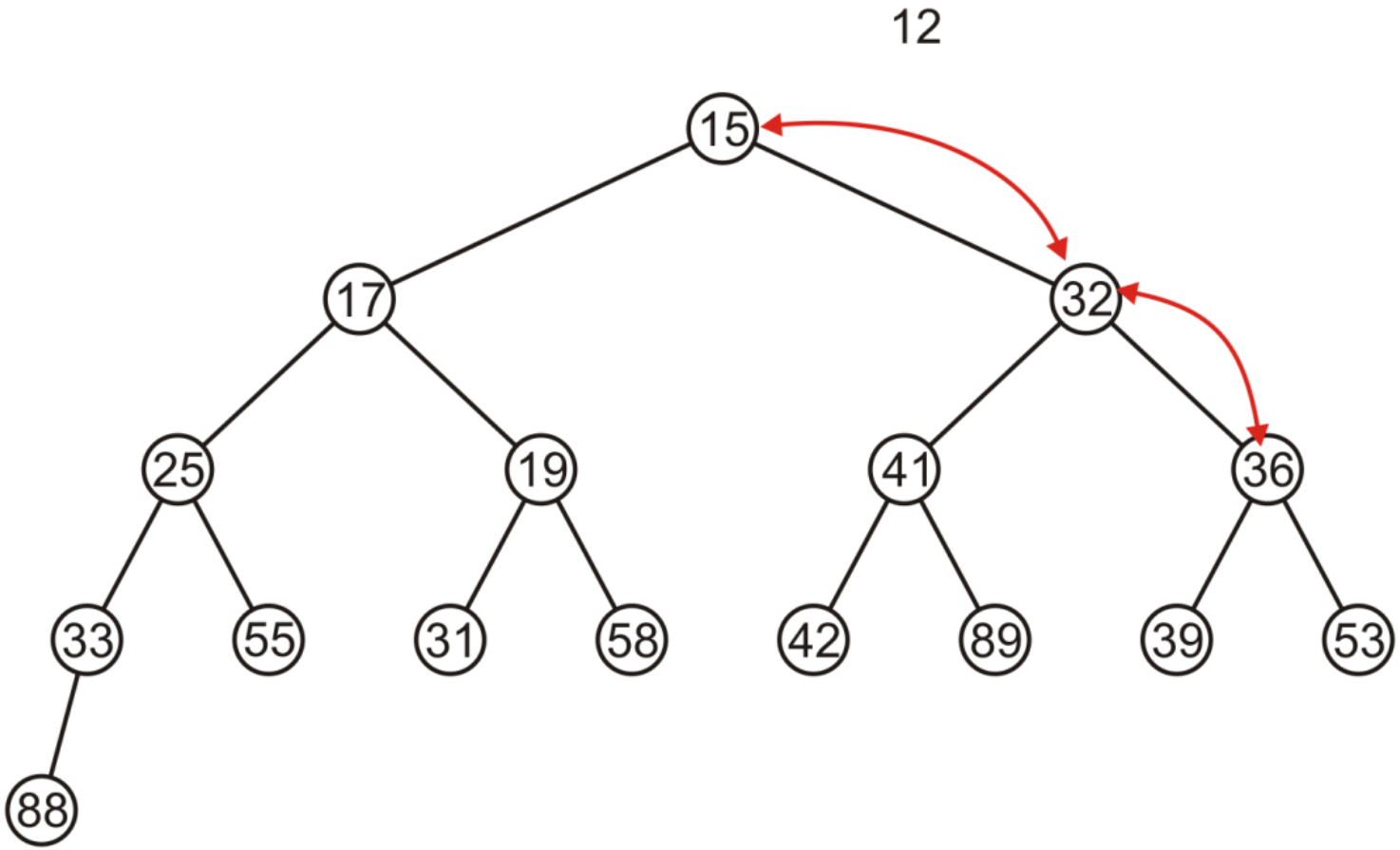
`deleteMin` – Example

- Copy the last entry in the heap to the root

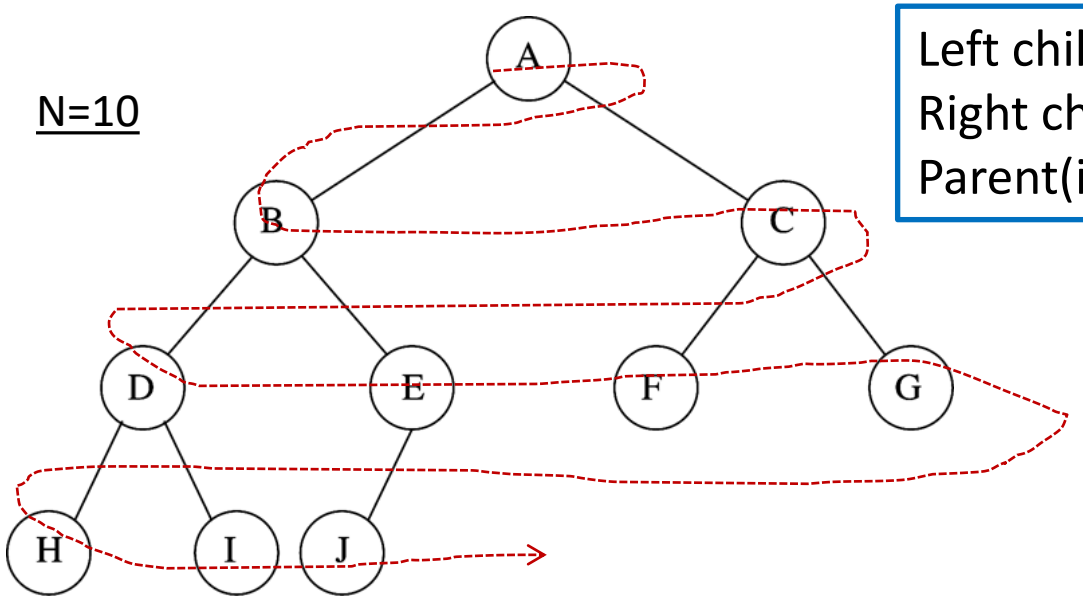


`deleteMin` – Example

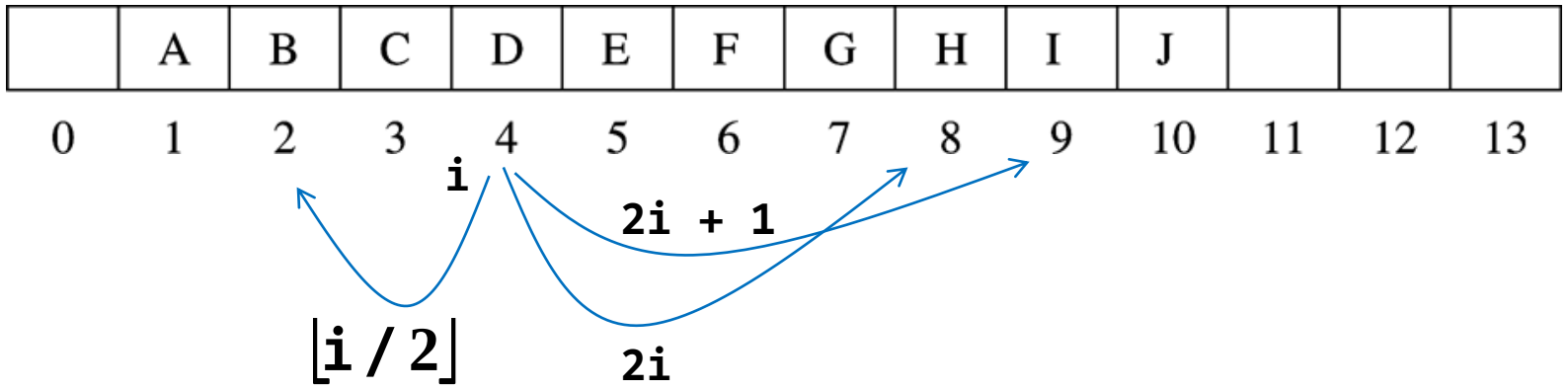
- Percolate 36 down swapping it with the smallest of its children
  - Halt when both children are larger



Array-Based Implementation of Binary Tree

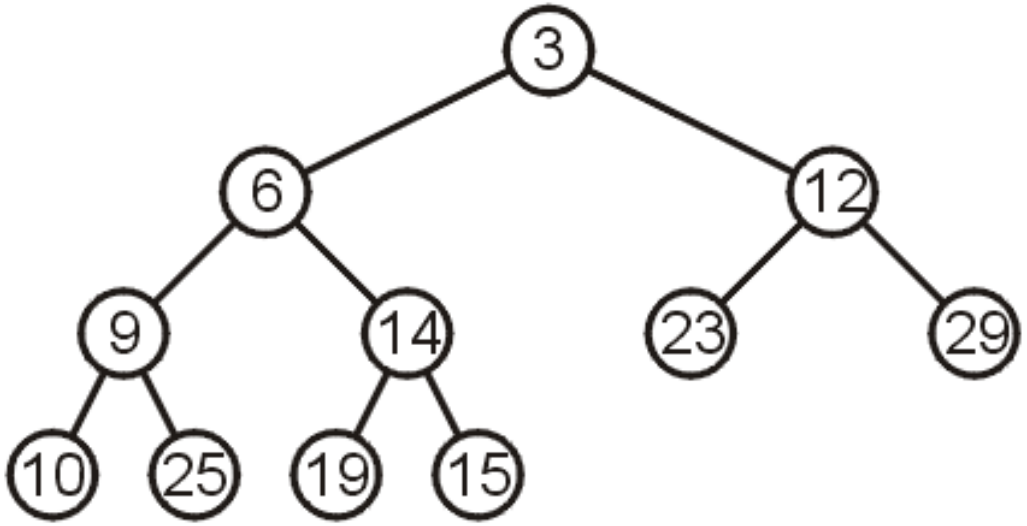


Left child(i) = at position 2i
Right child(i) = at position 2i + 1
Parent(i) = at position $\lfloor i / 2 \rfloor$



Array-Based Implementation Of Binary Heap

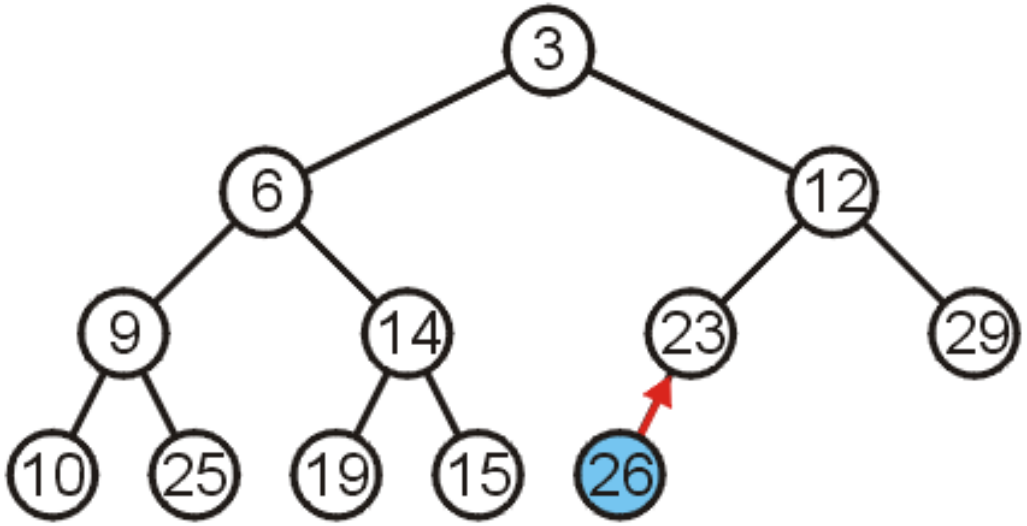
- Consider the following heap, both as a tree and in its array representation

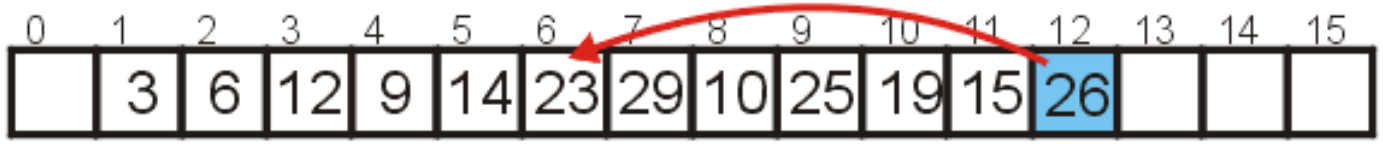


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	3	6	12	9	14	23	29	10	25	19	15				

Array-Based Implementation – `insert`

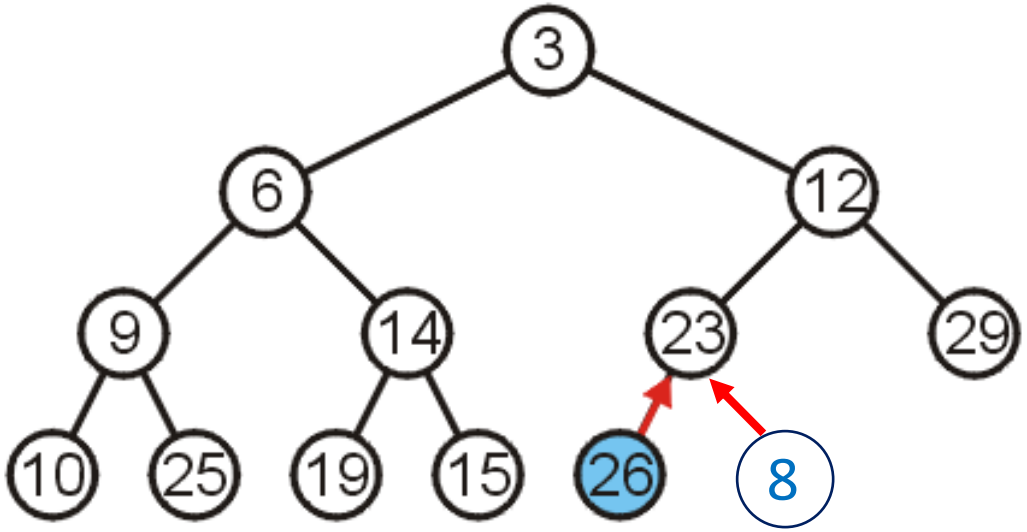
- Inserting 26 requires no changes

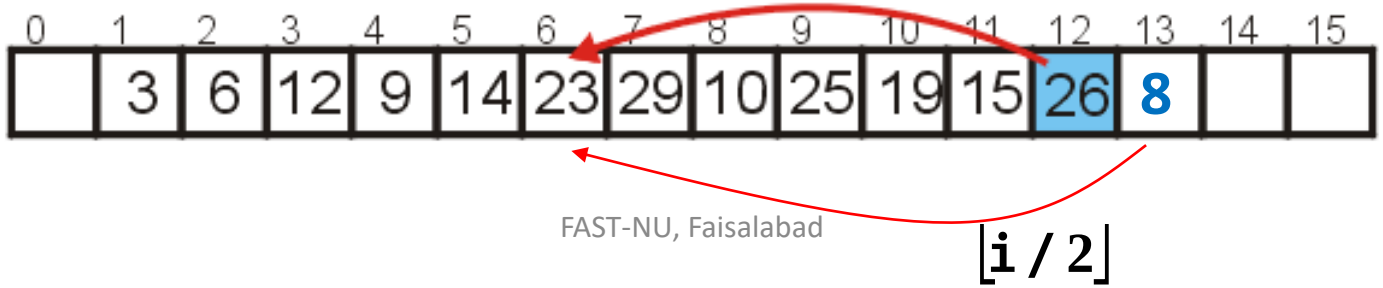




Array-Based Implementation – `insert`

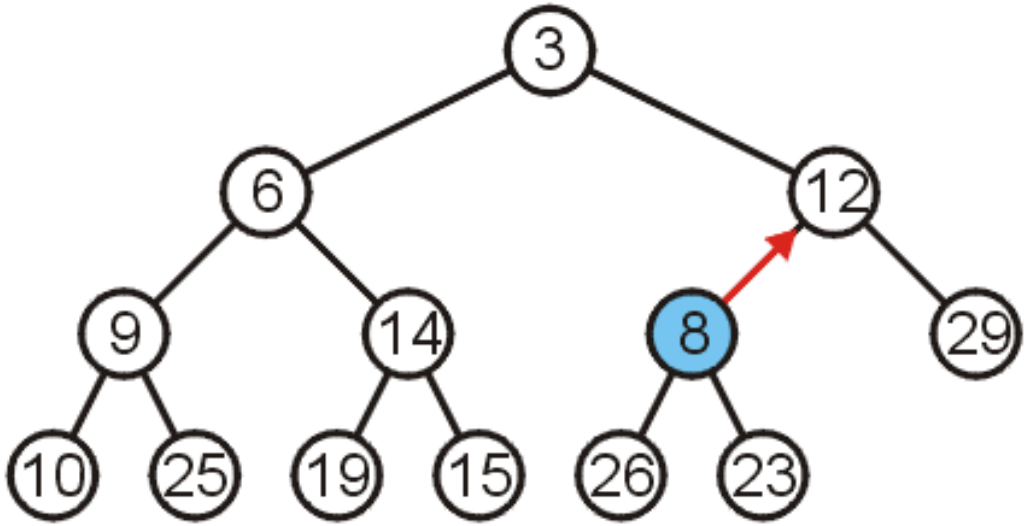
- Inserting 8 requires a few percolations
  - Swap 8 and 23

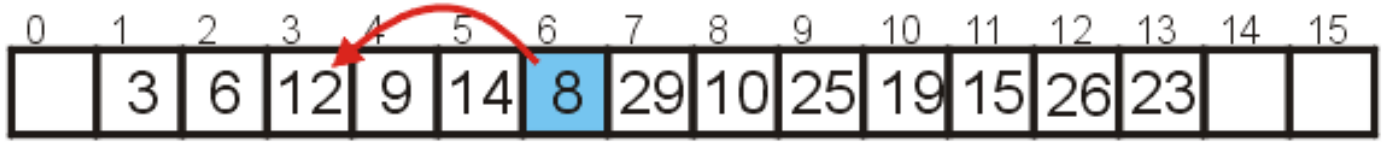




Array-Based Implementation – `insert`

- Swap 8 and 12





Array-Based Implementation – `insert`

- At this point, 8 is greater than its parent, so we are finished

